

# AI-Based IoT Botnet Detection System

Project Documentation & Source Code

## 1. Project Overview

A real-time, dark-themed cyber-security dashboard that simulates and visualizes IoT botnet detection. The system monitors CPU usage, network traffic and active devices, runs an anomaly-detection rule (CPU > 80% OR random anomaly), and triggers alerts plus mitigation actions when threats are detected.

## 2. Key Features

- Live metric cards: CPU Usage, Network Traffic (Mbps), Active Devices
- Real-time line chart of network traffic history (Chart.js)
- Detection panel showing Normal  or Botnet Detected  with confidence %
- Mitigation actions: Block IP, Isolate Device, Send Alert
- Animated logs feed and pop-up attack alert
- Simulate Traffic / Stop controls with fully animated state changes
- Responsive layout with collapsible sidebar

## 3. Technology Stack

| Technology                 | Purpose                                    |
|----------------------------|--------------------------------------------|
| React 18                   | UI library / component framework           |
| TypeScript                 | Static typing for safety & DX              |
| Vite                       | Build tool & dev server                    |
| Tailwind CSS               | Utility-first styling with semantic tokens |
| shadcn/ui                  | Reusable accessible UI primitives          |
| Framer Motion              | Animations & transitions                   |
| Chart.js + react-chartjs-2 | Real-time traffic line chart               |
| Lucide React               | Iconography                                |
| React Router               | Client-side routing                        |
| TanStack Query             | Async state management (ready for backend) |

# AI Prompts Used for IoT Botnet Detection Project

### Prompt 1

Suggest some unique AI/Deep Learning project ideas with patent potential in IoT security.

## **Prompt 2**

Suggest a patent title for AI-based botnet detection in IoT including multi-phase system and novel algorithms.

## **Prompt 3**

Write a patent draft in Indian format for the given IoT botnet detection system.

## **Prompt 4**

Expand the detailed description of the invention with multi-phase architecture.

## **Prompt 5**

Include fusion models at each phase (data, feature, model, decision).

## **Prompt 6**

Generate architecture diagram for the system.

## **Prompt 7**

Generate flowcharts for system phases.

## **Prompt 8**

Generate UI prototype for IoT botnet detection dashboard.

## **Prompt 9**

Build a web dashboard with metrics, graphs, detection alerts, and mitigation.

## **Prompt 10**

Build and deploy the dashboard and provide live URL.

## **Prompt 11**

Create simulation logic for IoT traffic and botnet detection.

## **Prompt 12**

Add charts to visualize anomalies and traffic.

## Prompt 13

Explain system architecture for viva.

### 4. Project Structure

```
src/
├── App.tsx                # Routes & providers
├── main.tsx               # React entry point
├── index.css              # Theme tokens + neon utilities
├── pages/
│   ├── Index.tsx         # Main dashboard page
│   └── NotFound.tsx
├── hooks/
│   └── useBotnetSimulation.ts # Simulation engine & state
├── components/dashboard/
│   ├── DashboardSidebar.tsx
│   ├── MetricCard.tsx
│   ├── TrafficChart.tsx
│   ├── DetectionPanel.tsx
│   ├── MitigationPanel.tsx
│   ├── LogsPanel.tsx
│   └── AttackAlert.tsx
```

### 5. Detection Logic

Each simulation tick (every 1.5s) generates randomized metrics. Anomaly is flagged when:

```
anomalyDetected = (cpu > 80) || (Math.random() > 0.75)
confidenceScore = anomalyDetected
  ? 60 + Math.random() * 39 // 60-99%
  : Math.random() * 30      // 5-30%
```

## 6. Source Code Files

All project source files are listed below in full.

### package.json

```
{
  "name": "vite_react_shadcn_ts",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "build:dev": "vite build --mode development",
    "lint": "eslint .",
    "preview": "vite preview",
    "test": "vitest run",
    "test:watch": "vitest"
  },
  "dependencies": {
    "@hookform/resolvers": "^3.10.0",
    "@radix-ui/react-accordion": "^1.2.11",
    "@radix-ui/react-alert-dialog": "^1.1.14",
    "@radix-ui/react-aspect-ratio": "^1.1.7",
    "@radix-ui/react-avatar": "^1.1.10",
    "@radix-ui/react-checkbox": "^1.3.2",
    "@radix-ui/react-collapsible": "^1.1.11",
    "@radix-ui/react-context-menu": "^2.2.15",
    "@radix-ui/react-dialog": "^1.1.14",
    "@radix-ui/react-dropdown-menu": "^2.1.15",
    "@radix-ui/react-hover-card": "^1.1.14",
    "@radix-ui/react-label": "^2.1.7",
    "@radix-ui/react-menubar": "^1.1.15",
    "@radix-ui/react-navigation-menu": "^1.2.13",
    "@radix-ui/react-popover": "^1.1.14",
    "@radix-ui/react-progress": "^1.1.7",
    "@radix-ui/react-radio-group": "^1.3.7",
    "@radix-ui/react-scroll-area": "^1.2.9",
    "@radix-ui/react-select": "^2.2.5",
    "@radix-ui/react-separator": "^1.1.7",
    "@radix-ui/react-slider": "^1.3.5",
    "@radix-ui/react-slot": "^1.2.3",
    "@radix-ui/react-switch": "^1.2.5",
    "@radix-ui/react-tabs": "^1.1.12",
    "@radix-ui/react-toast": "^1.2.14",
    "@radix-ui/react-toggle": "^1.1.9",
    "@radix-ui/react-toggle-group": "^1.1.10",
    "@radix-ui/react-tooltip": "^1.2.7",
    "@tanstack/react-query": "^5.83.0",
    "chart.js": "^4.5.1",
    "class-variance-authority": "^0.7.1",
    "clsx": "^2.1.1",
    "cmdk": "^1.1.1",
    "date-fns": "^3.6.0",
    "embla-carousel-react": "^8.6.0",
    "framer-motion": "^12.38.0",
    "input-otp": "^1.4.2",
    "lucide-react": "^1.7.0",
    "next-themes": "^0.3.0",
    "react": "^18.3.1",
    "react-chartjs-2": "^5.3.1",
    "react-day-picker": "^8.10.1",
  }
}
```

```
"react-dom": "^18.3.1",
"react-hook-form": "^7.61.1",
"react-resizable-panels": "^2.1.9",
"react-router-dom": "^6.30.1",
"recharts": "^2.15.4",
"sonner": "^1.7.4",
"tailwind-merge": "^2.6.0",
"tailwindcss-animate": "^1.0.7",
"vaul": "^0.9.9",
"zod": "^3.25.76"
},
"devDependencies": {
  "@eslint/js": "^9.32.0",
  "@playwright/test": "^1.57.0",
  "@tailwindcss/typography": "^0.5.16",
  "@testing-library/jest-dom": "^6.6.0",
  "@testing-library/react": "^16.0.0",
  "@types/node": "^22.16.5",
  "@types/react": "^18.3.23",
  "@types/react-dom": "^18.3.7",
  "@vitejs/plugin-react-swc": "^3.11.0",
  "autoprefixer": "^10.4.21",
  "eslint": "^9.32.0",
  "eslint-plugin-react-hooks": "^5.2.0",
  "eslint-plugin-react-refresh": "^0.4.20",
  "globals": "^15.15.0",
  "jsdom": "^20.0.3",
  "lovable-tagger": "^1.1.13",
  "postcss": "^8.5.6",
  "tailwindcss": "^3.4.17",
  "typescript": "^5.8.3",
  "typescript-eslint": "^8.38.0",
  "vite": "^5.4.19",
  "vitest": "^3.2.4"
}
}
```

## index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>IoT Botnet Detection Dashboard</title>
    <meta name="description" content="AI-powered IoT botnet detection and network threat monitoring dashboard" />
    <meta name="author" content="CyberShield" />
    <meta property="og:title" content="IoT Botnet Detection Dashboard" />
    <meta property="og:description" content="AI-powered IoT botnet detection and network threat monitoring" />
    <meta property="og:type" content="website" />
    <meta property="og:image" content="https://lovable.dev/opengraph-image-p98pqg.png" />

    <meta name="twitter:card" content="summary_large_image" />
    <meta name="twitter:site" content="@Lovable" />
    <meta name="twitter:image" content="https://lovable.dev/opengraph-image-p98pqg.png" />
  </head>

  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
```

## tailwind.config.ts

```
import type { Config } from "tailwindcss";

export default {
  darkMode: ["class"],
  content: [
    "./pages/**/*.ts,tsx",
    "./components/**/*.ts,tsx",
    "./app/**/*.ts,tsx",
    "./src/**/*.ts,tsx",
  ],
  prefix: "",
  theme: {
    container: {
      center: true,
      padding: "2rem",
      screens: {
        "2xl": "1400px",
      },
    },
    extend: {
      fontFamily: {
        sans: ["Inter", "system-ui", "sans-serif"],
        mono: ["JetBrains Mono", "monospace"],
      },
      colors: {
        border: "hsl(var(--border))",
        input: "hsl(var(--input))",
        ring: "hsl(var(--ring))",
        background: "hsl(var(--background))",
        foreground: "hsl(var(--foreground))",
        primary: {
          DEFAULT: "hsl(var(--primary))",
          foreground: "hsl(var(--primary-foreground))",
        },
        secondary: {
          DEFAULT: "hsl(var(--secondary))",
          foreground: "hsl(var(--secondary-foreground))",
        },
        destructive: {
          DEFAULT: "hsl(var(--destructive))",
          foreground: "hsl(var(--destructive-foreground))",
        },
        muted: {
          DEFAULT: "hsl(var(--muted))",
          foreground: "hsl(var(--muted-foreground))",
        },
        accent: {
          DEFAULT: "hsl(var(--accent))",
          foreground: "hsl(var(--accent-foreground))",
        },
        popover: {
          DEFAULT: "hsl(var(--popover))",
          foreground: "hsl(var(--popover-foreground))",
        },
        card: {
          DEFAULT: "hsl(var(--card))",
          foreground: "hsl(var(--card-foreground))",
        },
        sidebar: {
          DEFAULT: "hsl(var(--sidebar-background))",
          foreground: "hsl(var(--sidebar-foreground))",
          primary: "hsl(var(--sidebar-primary))",
          "primary-foreground": "hsl(var(--sidebar-primary-foreground))",
          accent: "hsl(var(--sidebar-accent))",
          "accent-foreground": "hsl(var(--sidebar-accent-foreground))",
          border: "hsl(var(--sidebar-border))",
        },
      },
    },
  },
}
```

```

    ring: "hsl(var(--sidebar-ring))",
  },
  neon: {
    blue: "hsl(var(--neon-blue))",
    cyan: "hsl(var(--neon-cyan))",
    green: "hsl(var(--neon-green))",
    red: "hsl(var(--neon-red))",
    orange: "hsl(var(--neon-orange))",
    purple: "hsl(var(--neon-purple))",
  },
},
borderRadius: {
  lg: "var(--radius)",
  md: "calc(var(--radius) - 2px)",
  sm: "calc(var(--radius) - 4px)",
},
keyframes: {
  "accordion-down": {
    from: {
      height: "0",
    },
    to: {
      height: "var(--radix-accordion-content-height)",
    },
  },
  "accordion-up": {
    from: {
      height: "var(--radix-accordion-content-height)",
    },
    to: {
      height: "0",
    },
  },
},
animation: {
  "accordion-down": "accordion-down 0.2s ease-out",
  "accordion-up": "accordion-up 0.2s ease-out",
},
},
},
plugins: [require("tailwindcss-animate")],
} satisfies Config;

```



## src/index.css

```
@tailwind base;
@tailwind components;
@tailwind utilities;

@import
url('https://fonts.googleapis.com/css2?family=JetBrains+Mono:wght@400;500;600;700&family=Inter:wght@300;400;500;600;700&display=swap');

@layer base {
  :root {
    --background: 222 47% 6%;
    --foreground: 210 40% 92%;

    --card: 222 47% 9%;
    --card-foreground: 210 40% 92%;

    --popover: 222 47% 9%;
    --popover-foreground: 210 40% 92%;

    --primary: 199 89% 48%;
    --primary-foreground: 222 47% 6%;

    --secondary: 222 47% 14%;
    --secondary-foreground: 210 40% 92%;

    --muted: 222 30% 16%;
    --muted-foreground: 215 20% 55%;

    --accent: 160 84% 39%;
    --accent-foreground: 222 47% 6%;

    --destructive: 0 72% 51%;
    --destructive-foreground: 210 40% 98%;

    --border: 222 30% 18%;
    --input: 222 30% 18%;
    --ring: 199 89% 48%;

    --radius: 0.75rem;

    --neon-blue: 199 89% 48%;
    --neon-cyan: 180 100% 50%;
    --neon-green: 160 84% 39%;
    --neon-red: 0 72% 51%;
    --neon-orange: 25 95% 53%;
    --neon-purple: 270 76% 55%;

    --sidebar-background: 222 47% 5%;
    --sidebar-foreground: 210 40% 85%;
    --sidebar-primary: 199 89% 48%;
    --sidebar-primary-foreground: 222 47% 6%;
    --sidebar-accent: 222 47% 12%;
    --sidebar-accent-foreground: 210 40% 92%;
    --sidebar-border: 222 30% 14%;
    --sidebar-ring: 199 89% 48%;
  }
}
```

```
@layer base {
  * {
    @apply border-border;
  }
  body {
    @apply bg-background text-foreground font-sans;
  }
}

@layer utilities {
  .glow-blue {
    box-shadow: 0 0 15px hsl(199 89% 48% / 0.3), 0 0 30px hsl(199 89% 48% / 0.1);
  }
  .glow-green {
    box-shadow: 0 0 15px hsl(160 84% 39% / 0.3), 0 0 30px hsl(160 84% 39% / 0.1);
  }
  .glow-red {
    box-shadow: 0 0 15px hsl(0 72% 51% / 0.4), 0 0 30px hsl(0 72% 51% / 0.15);
  }
  .text-glow-blue {
    text-shadow: 0 0 10px hsl(199 89% 48% / 0.5);
  }
  .text-glow-green {
    text-shadow: 0 0 10px hsl(160 84% 39% / 0.5);
  }
  .text-glow-red {
    text-shadow: 0 0 10px hsl(0 72% 51% / 0.5);
  }
  .cyber-grid {
    background-image:
      linear-gradient(hsl(199 89% 48% / 0.03) 1px, transparent 1px),
      linear-gradient(90deg, hsl(199 89% 48% / 0.03) 1px, transparent 1px);
    background-size: 40px 40px;
  }
}
```

## src/App.tsx

```
import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
import { BrowserRouter, Route, Routes } from "react-router-dom";
import { Toaster as Sonner } from "@components/ui/sonner";
import { Toaster } from "@components/ui/toaster";
import { TooltipProvider } from "@components/ui/tooltip";
import Index from "../pages/Index.tsx";
import NotFound from "../pages/NotFound.tsx";

const queryClient = new QueryClient();

const App = () => (
  <QueryClientProvider client={queryClient}>
    <TooltipProvider>
      <Toaster />
      <Sonner />
      <BrowserRouter>
        <Routes>
          <Route path="/" element={<Index />} />
          { /* ADD ALL CUSTOM ROUTES ABOVE THE CATCH-ALL "*" ROUTE */ }
          <Route path="*" element={<NotFound />} />
        </Routes>
      </BrowserRouter>
    </TooltipProvider>
  </QueryClientProvider>
);

export default App;
```

## src/pages/Index.tsx

```
import { useState, useEffect } from "react";
import { Cpu, Wifi, Monitor, Play, Square } from "lucide-react";
import { motion } from "framer-motion";
import { Button } from "@components/ui/button";
import DashboardSidebar from "@components/dashboard/DashboardSidebar";
import MetricCard from "@components/dashboard/MetricCard";
import TrafficChart from "@components/dashboard/TrafficChart";
import DetectionPanel from "@components/dashboard/DetectionPanel";
import MitigationPanel from "@components/dashboard/MitigationPanel";
import LogsPanel from "@components/dashboard/LogsPanel";
import AttackAlert from "@components/dashboard/AttackAlert";
import { useBotnetSimulation } from "@hooks/useBotnetSimulation";

export default function Index() {
  const [sidebarOpen, setSidebarOpen] = useState(true);
  const {
    state,
    attackAlert,
    startSimulation,
    stopSimulation,
    blockIP,
    isolateDevice,
    sendAlert,
    dismissAlert,
    setAttackAlert,
  } = useBotnetSimulation();

  // Auto-trigger alert when botnet detected
  useEffect(() => {
    if (state.anomalyDetected && state.isRunning) {
      setAttackAlert(true);
    }
  }, [state.anomalyDetected, state.isRunning, setAttackAlert]);

  return (
    <div className="min-h-screen bg-background cyber-grid">
      <DashboardSidebar isOpen={sidebarOpen} onToggle={() => setSidebarOpen(!sidebarOpen)} />
      <AttackAlert show={attackAlert} onDismiss={dismissAlert} />

      <motion.main
        animate={{ marginLeft: sidebarOpen ? 220 : 64 }}
        transition={{ duration: 0.3, ease: "easeInOut" }}
        className="min-h-screen p-6"
      >
        {/* Header */}
        <div className="flex flex-col sm:flex-row items-start sm:items-center justify-between mb-8 gap-4">
          <div>
            <h1 className="text-2xl font-bold text-foreground text-glow-blue">
              IoT Botnet Detection Dashboard
            </h1>
            <p className="text-sm text-muted-foreground mt-1">Real-time network threat monitoring</p>
          </div>
          <div className="flex gap-2">
            {!state.isRunning ? (
              <Button onClick={startSimulation} className="gap-2 bg-neon-green/20 text-neon-green border border-neon-green/30 hover:bg-neon-green/30">
                <Play className="h-4 w-4" /> Simulate Traffic
              </Button>
            ) : (

```

```

        <Button onClick={stopSimulation} variant="outline" className="gap-2 border-neon-red/30
text-neon-red hover:bg-neon-red/10">
          <Square className="h-4 w-4" /> Stop
        </Button>
      )}
    </div>
  </div>

  { /* Metrics */}
  <div className="grid grid-cols-1 sm:grid-cols-3 gap-4 mb-6">
    <MetricCard title="CPU Usage" value={state.cpu} unit="%" icon={Cpu} color={state.cpu > 80 ?
"red" : "blue"} delay={0.1} />
    <MetricCard title="Network Traffic" value={state.networkTraffic} unit="Mbps" icon={Wifi}
color="green" delay={0.15} />
    <MetricCard title="Active Devices" value={state.activeDevices} icon={Monitor}
color="orange" delay={0.2} />
  </div>

  { /* Chart */}
  <div className="mb-6">
    <TrafficChart data={state.trafficHistory} />
  </div>

  { /* Detection + Mitigation + Logs */}
  <div className="grid grid-cols-1 lg:grid-cols-3 gap-4">
    <DetectionPanel detected={state.anomalyDetected} confidence={state.confidenceScore} />
    <MitigationPanel onBlockIP={blockIP} onIsolateDevice={isolateDevice}
onSendAlert={sendAlert} />
    <LogsPanel logs={state.logs} />
  </div>
</motion.main>
</div>
);
}

```

## src/hooks/useBotnetSimulation.ts

```
import { useState, useCallback, useRef } from "react";

export interface SimulationState {
  cpu: number;
  networkTraffic: number;
  activeDevices: number;
  anomalyDetected: boolean;
  confidenceScore: number;
  trafficHistory: number[];
  logs: LogEntry[];
  isRunning: boolean;
}

export interface LogEntry {
  id: string;
  timestamp: Date;
  message: string;
  type: "info" | "warning" | "danger";
}

const initialState: SimulationState = {
  cpu: 32,
  networkTraffic: 245,
  activeDevices: 48,
  anomalyDetected: false,
  confidenceScore: 12,
  trafficHistory: Array.from({ length: 20 }, () => Math.random() * 300 + 100),
  logs: [
    { id: "1", timestamp: new Date(), message: "System initialized", type: "info" },
    { id: "2", timestamp: new Date(), message: "Network monitoring active", type: "info" },
  ],
  isRunning: false,
};

export function useBotnetSimulation() {
  const [state, setState] = useState<SimulationState>(initialState);
  const intervalRef = useRef<ReturnType<typeof setInterval> | null>(null);
  const [attackAlert, setAttackAlert] = useState(false);

  const addLog = useCallback((message: string, type: LogEntry["type"]) => {
    setState((prev) => ({
      ...prev,
      logs: [
        { id: Date.now().toString(), timestamp: new Date(), message, type },
        ...prev.logs.slice(0, 49),
      ],
    }));
  }, []);

  const simulateTick = useCallback(() => {
    setState((prev) => {
      const cpuSpike = Math.random() > 0.7;
      const anomalyChance = Math.random() > 0.75;
      const cpu = cpuSpike
        ? Math.min(99, prev.cpu + Math.random() * 40)
        : Math.max(10, prev.cpu + (Math.random() - 0.5) * 20);
      const networkTraffic = Math.max(50, prev.networkTraffic + (Math.random() - 0.4) * 150);
      const activeDevices = Math.max(10, Math.floor(prev.activeDevices + (Math.random() - 0.5) * 8));
      const anomalyDetected = cpu > 80 || anomalyChance;
      const confidenceScore = anomalyDetected
        ? Math.min(99, 60 + Math.random() * 39)
    });
  });
}
```

```

      : Math.max(5, Math.random() * 30);

const trafficHistory = [...prev.trafficHistory.slice(1), networkTraffic];

return {
  ...prev,
  cpu: Math.round(cpu),
  networkTraffic: Math.round(networkTraffic),
  activeDevices,
  anomalyDetected,
  confidenceScore: Math.round(confidenceScore),
  trafficHistory,
};
});
}, []);

const startSimulation = useCallback(() => {
  if (intervalRef.current) return;
  setState((prev) => ({ ...prev, isRunning: true }));
  addLog("Simulation started – monitoring traffic", "info");
  intervalRef.current = setInterval(() => {
    simulateTick();
  }, 1500);
}, [simulateTick, addLog]);

const stopSimulation = useCallback(() => {
  if (intervalRef.current) {
    clearInterval(intervalRef.current);
    intervalRef.current = null;
  }
  setState((prev) => ({ ...prev, isRunning: false }));
  addLog("Simulation stopped", "info");
}, [addLog]);

const blockIP = useCallback(() => {
  addLog(`Blocked suspicious IP: 192.168.${Math.floor(Math.random() * 255)}${Math.floor(Math.random() * 255)}`, "warning");
}, [addLog]);

const isolateDevice = useCallback(() => {
  addLog(`Device IoT-${Math.floor(Math.random() * 999).toString().padStart(3, "0")} isolated from network`, "warning");
  setState((prev) => ({ ...prev, activeDevices: Math.max(1, prev.activeDevices - 1) }));
}, [addLog]);

const sendAlert = useCallback(() => {
  addLog("Alert sent to security team", "danger");
  setAttackAlert(true);
  setTimeout(() => setAttackAlert(false), 3000);
}, [addLog]);

const dismissAlert = useCallback(() => setAttackAlert(false), []);

return {
  state,
  attackAlert,
  startSimulation,
  stopSimulation,
  blockIP,
  isolateDevice,
  sendAlert,
  dismissAlert,

```

```
    setAttackAlert,  
  };  
}
```



## src/components/dashboard/DashboardSidebar.tsx

```
import { Shield, Activity, Monitor, AlertTriangle, FileText, Radio } from "lucide-react";
import { motion } from "framer-motion";

interface Props {
  isOpen: boolean;
  onToggle: () => void;
}

const navItems = [
  { icon: Monitor, label: "Overview" },
  { icon: Activity, label: "Traffic" },
  { icon: AlertTriangle, label: "Detections" },
  { icon: Radio, label: "Devices" },
  { icon: FileText, label: "Logs" },
];

export default function DashboardSidebar({ isOpen, onToggle }: Props) {
  return (
    <motion.aside
      initial={false}
      animate={{ width: isOpen ? 220 : 64 }}
      transition={{ duration: 0.3, ease: "easeInOut" }}
      className="fixed left-0 top-0 h-full z-30 flex flex-col border-r border-border bg-sidebar
overflow-hidden"
    >
      <div className="flex items-center gap-3 px-4 h-16 border-b border-border">
        <button onClick={onToggle} className="p-1 rounded hover:bg-sidebar-accent transition-colors">
          <Shield className="h-6 w-6 text-primary" />
        </button>
        {isOpen && (
          <motion.span
            initial={{ opacity: 0 }}
            animate={{ opacity: 1 }}
            className="text-sm font-semibold text-sidebar-foreground whitespace-nowrap"
          >
            CyberShield
          </motion.span>
        )}
      </div>

      <nav className="flex-1 py-4 space-y-1 px-2">
        {navItems.map((item, i) => (
          <button
            key={item.label}
            className={`w-full flex items-center gap-3 px-3 py-2.5 rounded-md text-sidebar-foreground
hover:bg-sidebar-accent hover:text-primary transition-colors ${i === 0 ? "bg-sidebar-accent text-
primary" : ""}`}
          >
            <item.icon className="h-5 w-5 shrink-0" />
            {isOpen && (
              <motion.span
                initial={{ opacity: 0 }}
                animate={{ opacity: 1 }}
                className="text-sm whitespace-nowrap"
              >
                {item.label}
              </motion.span>
            )}
          </button>
        ))}
      </nav>
    </motion.aside>
  );
}
```

```
<div className="px-3 py-4 border-t border-border">
  <div className="flex items-center gap-2">
    <div className="h-2 w-2 rounded-full bg-neon-green animate-pulse" />
    {isOpen && <span className="text-xs text-muted-foreground">System Online</span>}
  </div>
</div>
</motion.aside>
);
}
```

## src/components/dashboard/MetricCard.tsx

```
import { motion } from "framer-motion";
import type { LucideIcon } from "lucide-react";

interface Props {
  title: string;
  value: string | number;
  unit?: string;
  icon: LucideIcon;
  color: "blue" | "green" | "red" | "orange";
  delay?: number;
}

const colorMap = {
  blue: "text-neon-blue glow-blue border-neon-blue/20",
  green: "text-neon-green glow-green border-neon-green/20",
  red: "text-neon-red glow-red border-neon-red/20",
  orange: "text-neon-orange border-neon-orange/20",
};

const iconBg = {
  blue: "bg-neon-blue/10",
  green: "bg-neon-green/10",
  red: "bg-neon-red/10",
  orange: "bg-neon-orange/10",
};

export default function MetricCard({ title, value, unit, icon: Icon, color, delay = 0 }: Props) {
  return (
    <motion.div
      initial={{ opacity: 0, y: 20 }}
      animate={{ opacity: 1, y: 0 }}
      transition={{ duration: 0.5, delay }}
      className={`rounded-xl border bg-card p-5 ${colorMap[color]}`
    >
      <div className="flex items-center justify-between mb-3">
        <span className="text-xs font-medium uppercase tracking-wider text-muted-foreground">{title}</span>
        <div className={`p-2 rounded-lg ${iconBg[color]}`>
          <Icon className={`h-4 w-4 ${colorMap[color].split(" ")[0]} ` />
        </div>
      </div>
      <div className="flex items-baseline gap-1">
        <span className="text-3xl font-bold font-mono">{value}</span>
        {unit && <span className="text-sm text-muted-foreground">{unit}</span>}
      </div>
    </motion.div>
  );
}
```

## src/components/dashboard/TrafficChart.tsx

```
import { Line } from "react-chartjs-2";
import {
  Chart as ChartJS,
  CategoryScale,
  LinearScale,
  PointElement,
  LineElement,
  Title,
  Tooltip,
  Filler,
} from "chart.js";
import { motion } from "framer-motion";

ChartJS.register(CategoryScale, LinearScale, PointElement, LineElement, Title, Tooltip, Filler);

interface Props {
  data: number[];
}

export default function TrafficChart({ data }: Props) {
  const labels = data.map((_, i) => `${i}s`);

  const chartData = {
    labels,
    datasets: [
      {
        label: "Traffic (Mbps)",
        data,
        borderColor: "hsl(199, 89%, 48%)",
        backgroundColor: "hsla(199, 89%, 48%, 0.08)",
        fill: true,
        tension: 0.4,
        pointRadius: 0,
        pointHoverRadius: 4,
        borderWidth: 2,
      },
    ],
  };

  const options = {
    responsive: true,
    maintainAspectRatio: false,
    scales: {
      x: {
        grid: { color: "hsla(199, 89%, 48%, 0.06)" },
        ticks: { color: "hsl(215, 20%, 55%)", font: { size: 10 } },
      },
      y: {
        grid: { color: "hsla(199, 89%, 48%, 0.06)" },
        ticks: { color: "hsl(215, 20%, 55%)", font: { size: 10 } },
      },
    },
    plugins: {
      tooltip: {
        backgroundColor: "hsl(222, 47%, 9%)",
        borderColor: "hsl(199, 89%, 48%)",
        borderWidth: 1,
        titleColor: "hsl(210, 40%, 92%)",
        bodyColor: "hsl(210, 40%, 92%)",
      },
    },
  };
}
```

```
};

return (
  <motion.div
    initial={{ opacity: 0, y: 20 }}
    animate={{ opacity: 1, y: 0 }}
    transition={{ duration: 0.5, delay: 0.3 }}
    className="rounded-xl border border-border bg-card p-5"
  >
    <h3 className="text-sm font-semibold text-foreground mb-4">Network Traffic Over Time</h3>
    <div className="h-64">
      <Line data={chartData} options={options} />
    </div>
  </motion.div>
);
}
```

## src/components/dashboard/DetectionPanel.tsx

```
import { motion, AnimatePresence } from "framer-motion";
import { ShieldCheck, ShieldAlert } from "lucide-react";

interface Props {
  detected: boolean;
  confidence: number;
}

export default function DetectionPanel({ detected, confidence }: Props) {
  return (
    <motion.div
      initial={{ opacity: 0, y: 20 }}
      animate={{ opacity: 1, y: 0 }}
      transition={{ duration: 0.5, delay: 0.4 }}
      className={`rounded-xl border p-5 ${
        detected
          ? "border-neon-red/30 bg-neon-red/5 glow-red"
          : "border-neon-green/30 bg-neon-green/5 glow-green"
      }`}
    >
      <h3 className="text-sm font-semibold text-foreground mb-4">Detection Status</h3>

      <AnimatePresence mode="wait">
        <motion.div
          key={detected ? "danger" : "safe"}
          initial={{ scale: 0.8, opacity: 0 }}
          animate={{ scale: 1, opacity: 1 }}
          exit={{ scale: 0.8, opacity: 0 }}
          transition={{ duration: 0.3 }}
          className="flex flex-col items-center py-4 gap-3"
        >
          {detected ? (
            <>
              <ShieldAlert className="h-12 w-12 text-neon-red animate-pulse" />
              <span className="text-lg font-bold text-neon-red text-glow-red">Botnet Detected
                </span>
            </>
          ) : (
            <>
              <ShieldCheck className="h-12 w-12 text-neon-green" />
              <span className="text-lg font-bold text-neon-green text-glow-green">Normal
                </span>
            </>
          )}
        </motion.div>
      </AnimatePresence>
      <div className="w-full mt-2">
        <div className="flex justify-between text-xs text-muted-foreground mb-1">
          <span>Confidence</span>
          <span className="font-mono">{confidence}%</span>
        </div>
        <div className="h-2 rounded-full bg-muted overflow-hidden">
          <motion.div
            className={`h-full rounded-full ${detected ? "bg-neon-red" : "bg-neon-green"}`}
            initial={{ width: 0 }}
            animate={{ width: `${confidence}%` }}
            transition={{ duration: 0.5 }}
          />
        </div>
      </div>
    </motion.div>
  );
}
```

}

## src/components/dashboard/MitigationPanel.tsx

```
import { Ban, Unplug, Bell } from "lucide-react";
import { motion } from "framer-motion";
import { Button } from "@components/ui/button";

interface Props {
  onBlockIP: () => void;
  onIsolateDevice: () => void;
  onSendAlert: () => void;
}

export default function MitigationPanel({ onBlockIP, onIsolateDevice, onSendAlert }: Props) {
  return (
    <motion.div
      initial={{ opacity: 0, y: 20 }}
      animate={{ opacity: 1, y: 0 }}
      transition={{ duration: 0.5, delay: 0.5 }}
      className="rounded-xl border border-border bg-card p-5"
    >
      <h3 className="text-sm font-semibold text-foreground mb-4">Mitigation Actions</h3>
      <div className="space-y-3">
        <Button
          onClick={onBlockIP}
          variant="outline"
          className="w-full justify-start gap-3 border-neon-red/30 text-neon-red hover:bg-neon-red/10
hover:text-neon-red"
        >
          <Ban className="h-4 w-4" /> Block IP
        </Button>
        <Button
          onClick={onIsolateDevice}
          variant="outline"
          className="w-full justify-start gap-3 border-neon-orange/30 text-neon-orange hover:bg-neon-
orange/10 hover:text-neon-orange"
        >
          <Unplug className="h-4 w-4" /> Isolate Device
        </Button>
        <Button
          onClick={onSendAlert}
          variant="outline"
          className="w-full justify-start gap-3 border-neon-blue/30 text-neon-blue hover:bg-neon-
blue/10 hover:text-neon-blue"
        >
          <Bell className="h-4 w-4" /> Send Alert
        </Button>
      </div>
    </motion.div>
  );
}
```



## src/components/dashboard/LogsPanel.tsx

```
import { motion, AnimatePresence } from "framer-motion";
import type { LogEntry } from "@/hooks/useBotnetSimulation";

interface Props {
  logs: LogEntry[];
}

const typeColor = {
  info: "text-neon-blue",
  warning: "text-neon-orange",
  danger: "text-neon-red",
};

const dotColor = {
  info: "bg-neon-blue",
  warning: "bg-neon-orange",
  danger: "bg-neon-red",
};

export default function LogsPanel({ logs }: Props) {
  return (
    <motion.div
      initial={{ opacity: 0, y: 20 }}
      animate={{ opacity: 1, y: 0 }}
      transition={{ duration: 0.5, delay: 0.6 }}
      className="rounded-xl border border-border bg-card p-5"
    >
      <h3 className="text-sm font-semibold text-foreground mb-4">Recent Events</h3>
      <div className="space-y-2 max-h-64 overflow-y-auto pr-1 scrollbar-thin">
        <AnimatePresence initial={false}>
          {logs.slice(0, 15).map((log) => (
            <motion.div
              key={log.id}
              initial={{ opacity: 0, x: -20 }}
              animate={{ opacity: 1, x: 0 }}
              exit={{ opacity: 0 }}
              transition={{ duration: 0.2 }}
              className="flex items-start gap-3 py-2 px-3 rounded-lg bg-muted/30 font-mono text-xs"
            >
              <div className={`h-2 w-2 rounded-full mt-1.5 shrink-0 ${dotColor[log.type]} `} />
              <div className="flex-1 min-w-0">
                <span className="text-muted-foreground">
                  {log.timestamp.toLocaleTimeString()}
                </span>
                <span className={`ml-2 ${typeColor[log.type]} `}>{log.message}</span>
              </div>
            </motion.div>
          ))}
        </AnimatePresence>
      </div>
    </motion.div>
  );
}
```

## src/components/dashboard/AttackAlert.tsx

```
import { motion, AnimatePresence } from "framer-motion";
import { AlertTriangle, X } from "lucide-react";

interface Props {
  show: boolean;
  onDismiss: () => void;
}

export default function AttackAlert({ show, onDismiss }: Props) {
  return (
    <AnimatePresence>
      {show && (
        <motion.div
          initial={{ opacity: 0, y: -50, scale: 0.9 }}
          animate={{ opacity: 1, y: 0, scale: 1 }}
          exit={{ opacity: 0, y: -50, scale: 0.9 }}
          className="fixed top-4 right-4 z-50 max-w-sm w-full rounded-xl border border-neon-red/40
bg-card p-4 glow-red"
        >
          <div className="flex items-start gap-3">
            <AlertTriangle className="h-6 w-6 text-neon-red animate-pulse shrink-0" />
            <div className="flex-1">
              <p className="font-semibold text-neon-red text-sm">⚠ Botnet Attack Detected</p>
              <p className="text-xs text-muted-foreground mt-1">
                Anomalous traffic pattern identified. Immediate action recommended.
              </p>
            </div>
            <button onClick={onDismiss} className="text-muted-foreground hover:text-foreground">
              <X className="h-4 w-4" />
            </button>
          </div>
        </motion.div>
      )}
    </AnimatePresence>
  );
}
```

